Lab 5.2 - Custom Topologies with the Mininet Python API

Task1

```
osboxes@osboxes:~/netprog-2026/lab05_custom_topo$ sudo python3
custom_topo.py
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
*** Starting CLI:
mininet>
```

Task2

```
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s2) (s1, s2)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 2 switches
s1 s2 ...
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
*** Starting CLI:
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s2-eth1
s1 lo:  s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4
:s2-eth2
s2 lo:  s2-eth1:h4-eth0 s2-eth2:s1-eth4
c0
mininet>
```

Task3

```
mininet> iperf h1 h4
*** Iperf: testing TCP bandwidth between h1 and h4
*** Results: ['4.7 Mbits/sec', '4.6 Mbits/sec']
```

Task4

```
mininet> h1 ping -c 5 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=101 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=100 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=100 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=100 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=100 ms

--- 10.0.0.4 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4002ms
rtt min/avg/max/mdev = 100.275/100.474/100.908/0.223 ms
mininet> h1 ping -c 5 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.698 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.038 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.041 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.035 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.049 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4072ms
rtt min/avg/max/mdev = 0.035/0.172/0.698/0.262 ms
mininet>
```

Task 5

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 X
h3 -> h1 h2 X
h4 -> h1 h2 X
*** Results: 25% dropped (9/12 received)
```

Task6

Answer: Defining a network topology using Python offers greater structural flexibility. For instance, with `CustomTopo`, you can configure switch `s1` with three hosts while switch `s2` has only one setup impossible with standard single, linear, or tree topologies, which require an equal number of hosts per switch.

Furthermore, you can apply specific bandwidth, delay, and loss parameters to individual links; for example, the backbone link between `s1` and `s2` could be configured to simulate a WAN connection (5 Mbit/s, 50 ms latency, 10% loss) while other links operate at full speed. This allows for distinct performance characteristics within the same network, such as an RTT of 0.3 ms for the `h1`–`h2` path versus 107 ms for the `h1`–`h4` path.

Reflection

1. **Explain the role of the build() method in a Topo subclass when is it called, and by what?**
   Answer: The build() method is where we define the topology structure using addHost(), addSwitch(), and addLink(). It is automatically invoked by the Topo class constructor when an object—such as CustomTopo()—is instantiated (e.g., in the line Mininet(topo=CustomTopo())), so we do not need to call build() manually. The result of build() is merely a network "blueprint"; Mininet subsequently uses this blueprint to actually create the hosts, switches, and links.

2. **Why is link=TCLink required?**
   Answer: The addLink() method merely records bandwidth (bw), delay, and loss values in the blueprint. However, when creating the actual links, Mininet defaults to the standard Link class, which creates simple veth pairs and ignores these parameters without raising an error. In contrast, TCLink utilizes Linux traffic control (tc) on the actual interfaces specifically HTB for bandwidth limiting and netem for delay and loss simulation. Therefore, we must explicitly instruct Mininet to use TCLink for these parameters to take effect.

3. **Real-world scenarios that single, linear, or tree topologies cannot model**
   Answer: Consider a company with a headquarters and a branch office. The headquarters has numerous machines connected via a high-speed LAN, while the branch office has only a few. The two sites are connected by a slow WAN link characterized by latency and occasional packet loss. We can simulate this using a custom topology where the switches at each site support an unequal number of hosts and the WAN link is configured with specific bw, delay, and loss settings. Standard single, linear, or tree topologies cannot achieve this, as they require an equal number of hosts per switch and uniform characteristics across all links.